



**Universidad Nacional
Autónoma de México**



Facultad de Ingeniería

**Laboratorio de Arquitectura y Organización
de Computadoras**

Grupo: 07 - Semestre: 2027-1

Reporte Practica 4

Construcción de Máquinas de estados Usando Memorias
Direccionamiento Entrada - Estado

Fecha de Entrega

2 de Octubre del 2026

Profesor

Ing. Julio Cesar Cruz Estrada

Integrantes:

Espinoza Matamoros Percival Ulises

Montiel Juárez Oscar Iván

Veraza García Amy Valentina

1. Objetivo

Familiarizar al alumno en el conocimiento de construcción de máquinas de estados usando direccionamiento de memorias con el método de direccionamiento entrada-estado.

2. Desarrollo

El método de direccionamiento entrada-estado se aplica a cartas ASM en las que cada estado consulta una sola variable de entrada. A diferencia del direccionamiento por trayectoria, la dirección de la ROM depende únicamente del estado presente. La palabra almacenada contiene el código de la entrada que debe probarse, dos ligas posibles y dos conjuntos de salidas. Un selector de entrada obtiene el valor de la variable indicada; con dicho valor, los selectores de liga y de salida escogen la alternativa falsa o verdadera.

Para esta práctica se implementó la carta ASM propia mostrada en la Figura 1. Las entradas se denominaron A , B y C , y las salidas se representaron por $S_3S_2S_1S_0$. La máquina tiene seis estados útiles, codificados de 000 a 101. El registro mantiene el estado presente y lo usa directamente como dirección de la ROM; por ello, cada estado requiere una única palabra de memoria.

2.1. Carta ASM y asignaciones binarias

La carta ASM se diseñó con tres decisiones: en S_0 se prueba A , en S_1 se prueba B y en S_2 se prueba C . Los estados S_3 , S_4 y S_5 no dependen de una entrada, por lo que su transición se fuerza al colocar las mismas ligas y salidas en las alternativas falsa y verdadera. Así, el valor seleccionado no modifica el resultado.

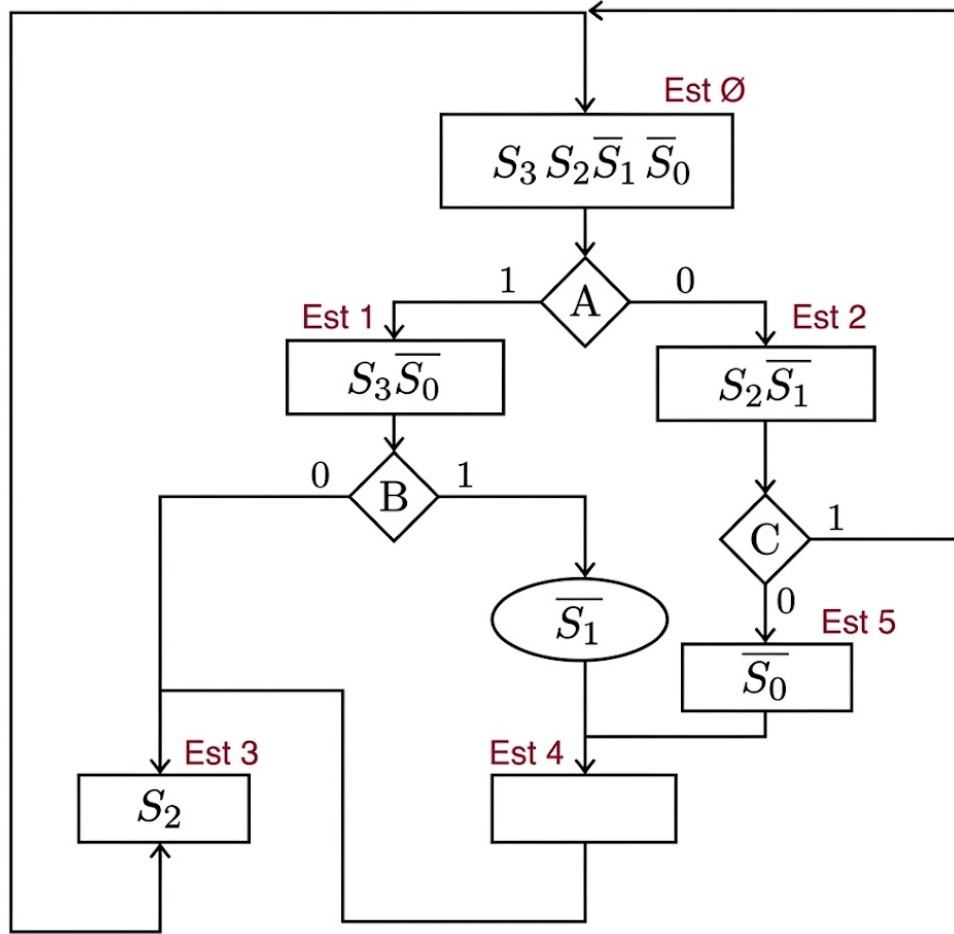


Figura 1: Carta ASM propia utilizada para el direccionamiento entrada-estado.

La asignación binaria de estados y de entradas es la siguiente:

Estado	S_0	S_1	S_2	S_3	S_4	S_5	Entrada	A	B	C
$Q_2Q_1Q_0$	000	001	010	011	100	101	P_1P_0	00	01	10

El código 11 de prueba queda disponible como valor auxiliar: en `mux_prueba.vhd` se resuelve como cero. En los estados sin decisión, la implementación utiliza 00, pero como sus campos falso y verdadero son idénticos, el selector es indiferente para la transición y las salidas.

2.2. Dimensionamiento de la memoria

Se requieren $q = \lceil \log_2 6 \rceil = 3$ bits para codificar los seis estados. Puesto que la dirección se forma sólo con el estado presente, la ROM tiene $n = q = 3$ bits de dirección. La palabra de

memoria se divide en cinco campos:

$$\underbrace{P_1P_0}_{2 \text{ bits de prueba}} \quad \underbrace{LF_2LF_1LF_0}_{3 \text{ bits de liga falsa}} \quad \underbrace{LV_2LV_1LV_0}_{3 \text{ bits de liga verdadera}} \quad \underbrace{SF_3SF_2SF_1SF_0}_{4 \text{ salidas falsas}} \quad \underbrace{SV_3SV_2SV_1SV_0}_{4 \text{ salidas verdaderas}}.$$

Por tanto,

$$m = 2 + 3 + 3 + 4 + 4 = 16 \text{ bits}, \quad \text{ROM} = 2^n \times m = 2^3 \times 16 = 8 \times 16 \text{ bits}.$$

La capacidad total requerida es de 128 bits. Las dos direcciones no utilizadas por los seis estados de la carta, 110 y 111, se programaron como recuperación segura hacia S_0 , con salidas 0011.

2.3. Contenido de la memoria

La Tabla 1 contiene la palabra de memoria de cada estado. P identifica la entrada que revisa `mux_prueba`; LF y LV son las ligas que se seleccionan para $Q_{sel} = 0$ y $Q_{sel} = 1$, respectivamente. De la misma forma, SF y SV son las salidas para cada alternativa. El asterisco indica un *don't care*.

Tabla 1: Tabla de direccionamiento entrada-estado y contenido de la ROM.

Estado	$Q_2Q_1Q_0$	Prueba P_1P_0	LF	LV	SF	SV
S_0	000	00	010	001	1100	1100
S_1	001	01	011	100	1010	1000
S_2	010	10	101	000	0101	0101
S_3	011	*	000	000	0111	0111
S_4	100	*	011	011	0011	0011
S_5	101	*	100	100	0010	0010
No definidas	110, 111	*	000	000	0011	0011

Por ejemplo, cuando el registro presenta 001, la ROM entrega 01 011 100 1010 1000. El selector de prueba interpreta 01 como la entrada B . Si $B = 0$, la máquina avanza a 011 y presenta 1010; si $B = 1$, avanza a 100 y presenta 1000. Esta sola palabra reemplaza las dos localidades que habría requerido el direccionamiento por trayectoria.

2.4. Arquitectura e implementación en VHDL

Las Figuras 2 y 3 muestran las capturas del diagrama de bloques implementado en Quartus. El estado presente se registra en cada flanco de reloj y direcciona la ROM. El bloque `div_datos` separa los cinco campos de la palabra; `mux_prueba` evalúa la entrada seleccionada y genera Q_{sel} . Finalmente, `mux_liga` y `mux_salidas` escogen, con la misma señal, la liga y las salidas que corresponden a la decisión de la carta ASM.

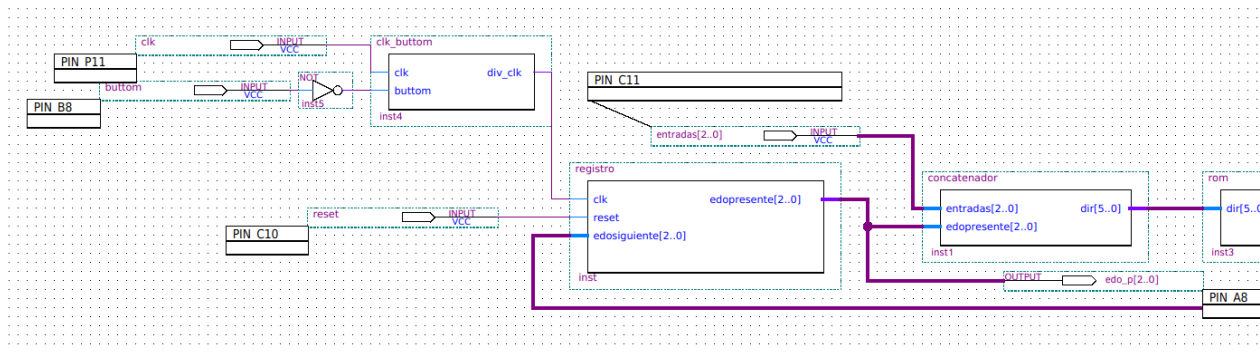


Figura 2: Primera parte del diagrama de bloques implementado en Quartus.

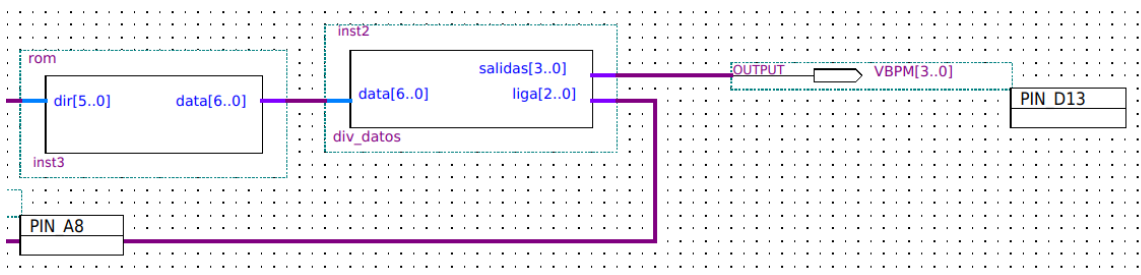


Figura 3: Segunda parte del diagrama de bloques implementado en Quartus.

Registro. El módulo `registro` almacena la liga seleccionada. Su reinicio es activo en bajo: si `reset='0'`, fuerza el estado 000; cuando `reset='1'`, actualiza el estado sólo en el flanco ascendente de `clk`. Esto asegura que cada transición de la máquina ocurra de manera síncrona.

ROM y división de datos. `rom.vhd` declara ocho palabras de 16 bits y usa el estado presente como dirección. `div_datos.vhd` no requiere reloj: separa los bits [15 : 14] como prueba, [13 : 11] como liga falsa, [10 : 8] como liga verdadera, [7 : 4] como salida falsa y [3 : 0] como salida verdadera.

Selección de entrada, liga y salidas. `mux_prueba` traduce los códigos 00, 01 y 10 a *A*, *B* y *C*. El resultado `Qsel` alimenta simultáneamente a `mux_liga` y `mux_salidas`: con cero se seleccionan *LF* y *SF*; con uno, *LV* y *SV*. Esta selección compartida preserva la correspondencia entre las ramas de la carta ASM, la transición de estado y las salidas.

Reloj por botón. `clk_button` cuenta pulsos del reloj de la tarjeta. Después de una pulsación, produce una señal `div_clk` que genera una transición controlada del registro. Sus estados internos de espera, conteo y finalización evitan que una pulsación prolongada produzca cambios no deseados de estado.

Listing 1: Registro del estado presente.

```

1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3  use IEEE.STD_LOGIC_ARITH.ALL;
4  use IEEE.STD_LOGIC_UNSIGNED.ALL;
5
6  entity registro is
7      port(
8          clk : in std_logic;
9          reset : in std_logic;
10         edosiguiente : in std_logic_vector(2 downto 0);
11         edopresente : out std_logic_vector(2 downto 0)
12     );
13 end registro;
14
15 architecture Behavioral of registro is
16
17     signal internal_value: std_logic_vector(2 downto 0) := B"000";
18
19     begin
20         process(clk, reset, edosiguiente)
21         begin
22             if reset = '0' then
23                 internal_value <= B"000";
24             elsif rising_edge(clk) then
25                 internal_value <= edosiguiente;
26             end if;
27         end process;
28
29         process(internal_value)
30         begin
31             edopresente <= internal_value;
32         end process;
33
34     end Behavioral;

```

Listing 2: ROM de ocho palabras para el método entrada-estado.

```

1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3  use IEEE.NUMERIC_STD.ALL;
4
5  entity rom is
6      port(
7          dir : in std_logic_vector(2 downto 0);
8          data : out std_logic_vector(15 downto 0)
9      );
10 end rom;
11
12 architecture Behavioral of rom is
13
14     type mem is array (0 to 7) of std_logic_vector(15 downto 0);
15     signal internal_mem : mem;
16
17     begin
18
19         -----
20         -- FORMATO:
21         --
22         -- data(15 downto 14) = Prueba
23         -- data(13 downto 11) = Liga Falsa
24         -- data(10 downto 8) = Liga Verdadera
25         -- data(7 downto 4) = Salida Falsa

```

```

26  -- data(3 downto 0) = Salida Verdadera
27  --
28  -- Prueba:
29  -- 00 = entrada A / I0
30  -- 01 = entrada B / I1
31  -- 10 = entrada C / I2
32  -----
33
34
35  -- Estado 0 = 000
36  -- prueba = 00
37  -- LF = 010
38  -- LV = 001
39  -- SF = 1100
40  -- SV = 1100
41  internal_mem(0) <=
42    "00" & "010" & "001" & "1100" & "1100";
43
44
45  -- Estado 1 = 001
46  -- prueba = 01
47  -- LF = 011
48  -- LV = 100
49  -- SF = 1010
50  -- SV = 1000
51  internal_mem(1) <=
52    "01" & "011" & "100" & "1010" & "1000";
53
54
55  -- Estado 2 = 010
56  -- prueba = 10
57  -- LF = 101
58  -- LV = 000
59  -- SF = 0101
60  -- SV = 0101
61  internal_mem(2) <=
62    "10" & "101" & "000" & "0101" & "0101";
63
64
65  -- Estado 3 = 011
66  -- prueba = ** (no importa)
67  -- LF = LV = 000
68  -- SF = SV = 0111
69  internal_mem(3) <=
70    "00" & "000" & "000" & "0111" & "0111";
71
72
73  -- Estado 4 = 100
74  -- prueba = ** (no importa)
75  -- LF = LV = 011
76  -- SF = SV = 0011
77  internal_mem(4) <=
78    "00" & "011" & "011" & "0011" & "0011";
79
80
81  -- Estado 5 = 101
82  -- prueba = ** (no importa)
83  -- LF = LV = 100
84  -- SF = SV = 0010
85  internal_mem(5) <=
86    "00" & "100" & "100" & "0010" & "0010";
87
88

```

```

89  -- Estado 6 = 110
90  -- No aparece especificado en la tabla.
91  -- Valor seguro: regresar al estado 000 con salida 0011.
92  internal_mem(6) <=
93      "00" & "000" & "000" & "0011" & "0011";
94
95
96  -- Estado 7 = 111
97  -- Sí aparece en la tabla
98  -- prueba = **
99  -- LF = LV = 000
100 -- SF = SV = 0011
101 internal_mem(7) <=
102     "00" & "000" & "000" & "0011" & "0011";
103
104
105 -----
106 -- Lectura de la ROM
107 -----
108 process(dir)
109 begin
110     data <= internal_mem(to_integer(unsigned(dir)));
111 end process;
112
113 end Behavioral;

```

Listing 3: Separación de los campos de la palabra de 16 bits.

```

1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3
4  entity div_datos is
5      port(
6          data      : in  std_logic_vector(15 downto 0);
7
8          prueba    : out std_logic_vector(1 downto 0);
9
10         liga_f     : out std_logic_vector(2 downto 0);
11         liga_v     : out std_logic_vector(2 downto 0);
12
13         salidas_f  : out std_logic_vector(3 downto 0);
14         salidas_v  : out std_logic_vector(3 downto 0)
15     );
16 end div_datos;
17
18 architecture Behavioral of div_datos is
19 begin
20
21     prueba    <= data(15 downto 14);
22
23     liga_f    <= data(13 downto 11);
24     liga_v    <= data(10 downto 8);
25
26     salidas_f <= data(7 downto 4);
27     salidas_v <= data(3 downto 0);
28
29 end Behavioral;

```

Listing 4: Selector de la entrada que debe probarse.

```

1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;

```

```

3 use IEEE.STD_LOGIC_ARITH.ALL;
4 use IEEE.STD_LOGIC_UNSIGNED.ALL;
5
6 entity mux_prueba is
7     port(
8         prueba: in std_logic_vector(1 downto 0);
9         I0: in std_logic;
10        I1: in std_logic;
11        I2: in std_logic;
12        Qsel: out std_logic
13    );
14 end mux_prueba;
15
16 architecture Behavioral of mux_prueba is
17 begin
18     process(prueba, I0, I1, I2)
19     begin
20         if prueba = "00" then
21             Qsel <= I0;
22         elsif prueba = "01" then
23             Qsel <= I1;
24         elsif prueba = "10" then
25             Qsel <= I2;
26         else
27             Qsel <= '0';
28         end if;
29     end process;
30 end Behavioral;

```

Listing 5: Selector de liga falsa o verdadera.

```

1 library IEEE;
2 use IEEE.STD_LOGIC_1164.ALL;
3 use IEEE.STD_LOGIC_ARITH.ALL;
4 use IEEE.STD_LOGIC_UNSIGNED.ALL;
5
6 entity mux_liga is
7     port(
8         Qsel: in std_logic;
9         LF: in std_logic_vector(2 downto 0);
10        LV: in std_logic_vector(2 downto 0);
11        Liga: out std_logic_vector(2 downto 0)
12    );
13 end mux_liga;
14
15 architecture Behavioral of mux_liga is
16 begin
17     process(Qsel, LF, LV)
18     begin
19         if Qsel = '0' then
20             Liga <= LF;
21         elsif Qsel = '1' then
22             Liga <= LV;
23         end if;
24     end process;
25 end Behavioral;

```

Listing 6: Selector de salidas falsas o verdaderas.

```

1 library IEEE;
2 use IEEE.STD_LOGIC_1164.ALL;
3

```

```

4  entity mux_salidas is
5      port (
6          Qsel : in  std_logic;
7          SF   : in  std_logic_vector(3 downto 0);
8          SV   : in  std_logic_vector(3 downto 0);
9          Salida : out std_logic_vector(3 downto 0)
10     );
11 end mux_salidas;
12
13 architecture Behavioral of mux_salidas is
14 begin
15
16     process(Qsel, SF, SV)
17     begin
18         if Qsel = '0' then
19             Salida <= SF;
20         else
21             Salida <= SV;
22         end if;
23     end process;
24
25 end Behavioral;

```

Listing 7: Acondicionamiento del reloj mediante botón.

```

1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3  use IEEE.STD_LOGIC_ARITH.ALL;
4  use IEEE.STD_LOGIC_UNSIGNED.ALL;
5
6  entity clk_button is
7      port(
8          clk : in std_logic;
9          button : in std_logic;
10         div_clk : out std_logic
11     );
12 end clk_button;
13
14 architecture Behavioral of clk_button is
15
16     CONSTANT STATE_START: std_logic_vector(1 downto 0) := "00";
17     CONSTANT STATE_COUNT: std_logic_vector(1 downto 0) := "01";
18     CONSTANT STATE_DONE: std_logic_vector(1 downto 0) := "10";
19
20     CONSTANT MAX_CLK_COUNT: std_logic_vector(27 downto 0) := X"17D7840"; -- 25 millones de
21         pulsos
22
23     signal state: std_logic_vector(1 downto 0) := "00";
24     signal cuenta: std_logic_vector(27 downto 0) := X"0000000";
25
26 begin
27     process(clk, button)
28     begin
29         if rising_edge(clk) then
30             case state is
31                 WHEN STATE_START =>
32                     if button = '1' then
33                         state <= STATE_COUNT;
34                     end if;
35                 WHEN STATE_COUNT =>
36                     if cuenta >= MAX_CLK_COUNT then
37                         state <= STATE_DONE;
38                     end if;

```

```

38         WHEN STATE_DONE =>
39             state <= STATE_START;
40         WHEN others =>
41             state <= STATE_START;
42     end case;
43 end if;
44 end process;
45
46 process(clk)
47 begin
48     if rising_edge(clk) then
49         if state = STATE_COUNT then
50             cuenta <= cuenta + 1;
51         else
52             cuenta <= X"0000000";
53         end if;
54     end if;
55 end process;
56
57 process(state)
58 begin
59     if state = STATE_COUNT then
60         div_clk <= '1';
61     else
62         div_clk <= '0';
63     end if;
64 end process;
65 end Behavioral;

```

3. Simulación y verificación

El proyecto incluye una simulación vectorial de Quartus para las señales de estado presente, salidas, **reset**, entradas y reloj. Durante la ejecución, **reset** se mantiene en uno para permitir el avance de la máquina; al llevarlo a cero, el registro regresa inmediatamente a S_0 . Las capturas muestran la evolución de los estados y las salidas ante las combinaciones de entrada aplicadas.

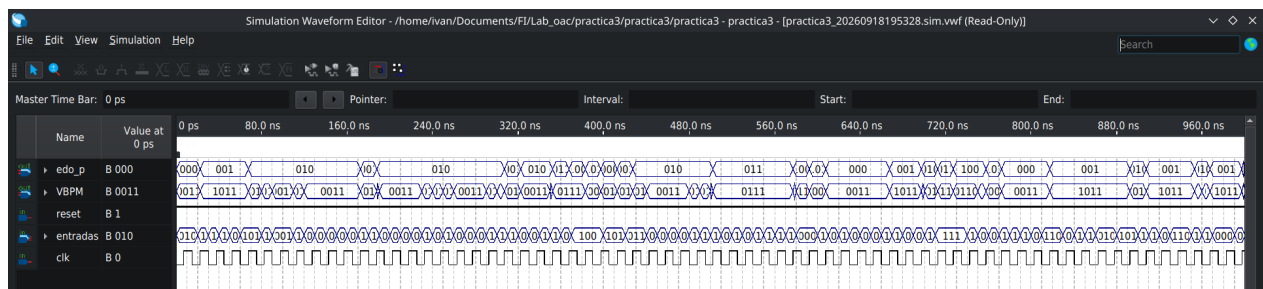


Figura 4: Primer tramo de la simulación en Quartus.

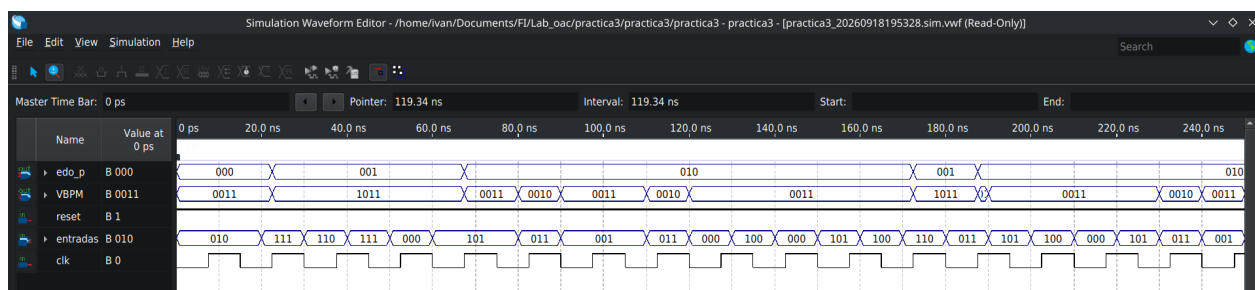


Figura 5: Detalle de las transiciones de estado y de las salidas durante la simulación.

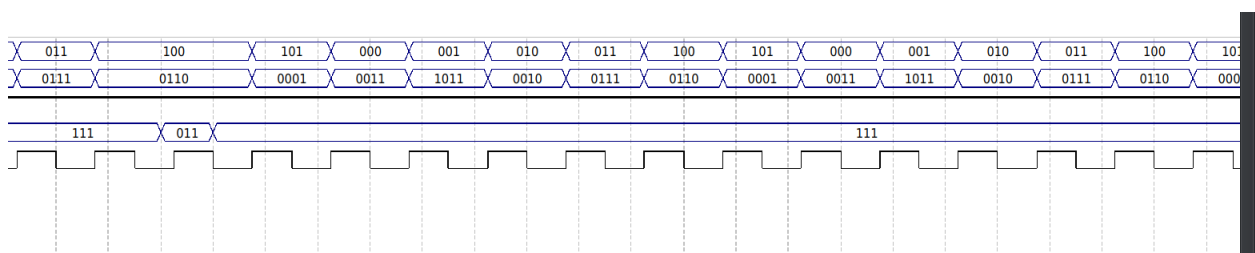


Figura 6: Tramo final de la simulación con las entradas forzadas.

4. Resultados en la tarjeta FPGA

Las fotografías muestran la implementación sobre la tarjeta DE10-Lite. Los LEDs permiten observar el código de `edo_presente` y el vector `salidas`. Se documentó S_0 , las dos condiciones de S_1 y los estados restantes de la carta; con ello se contrastan las ramas dependientes de entrada y las transiciones incondicionales programadas en la ROM.

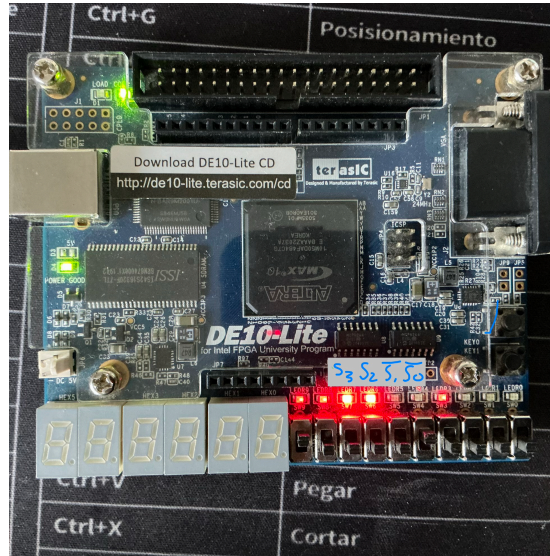
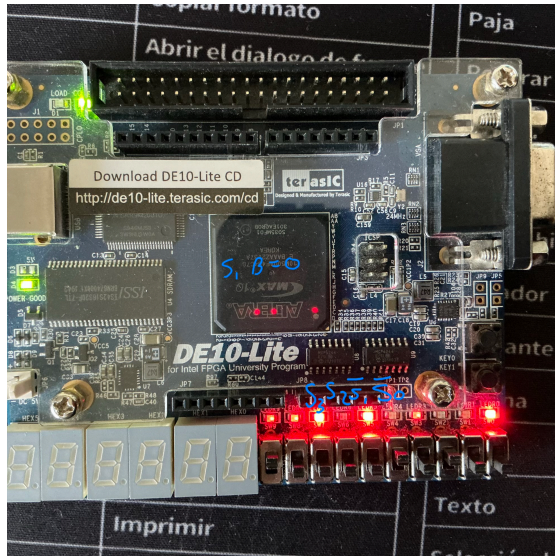
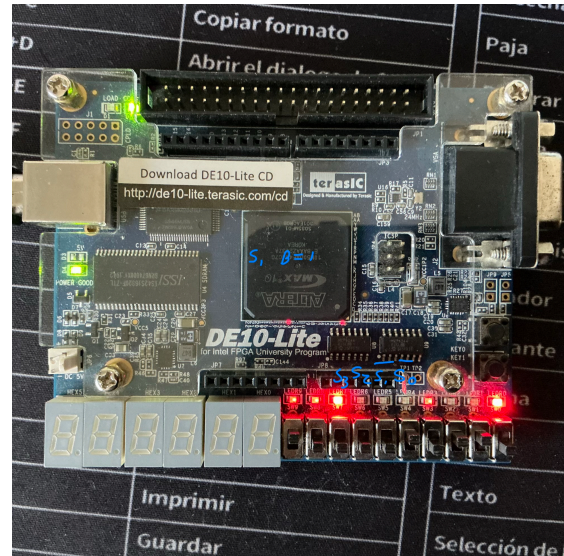


Figura 7: Evidencia en tarjeta del estado S_0 , que prueba la entrada A .

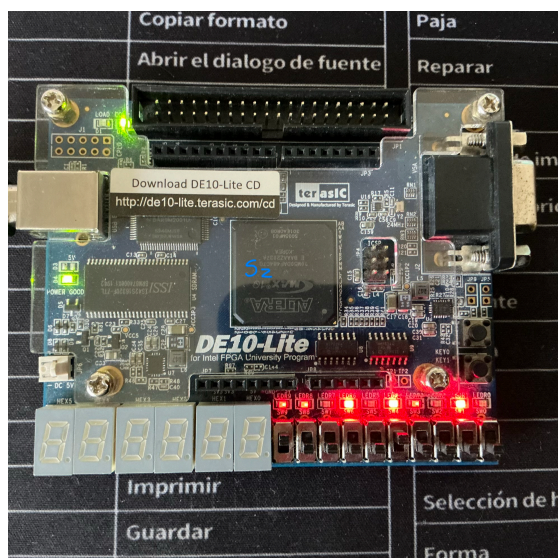


(a) S_1 con $B = 0$: se selecciona la liga falsa hacia S_3 .

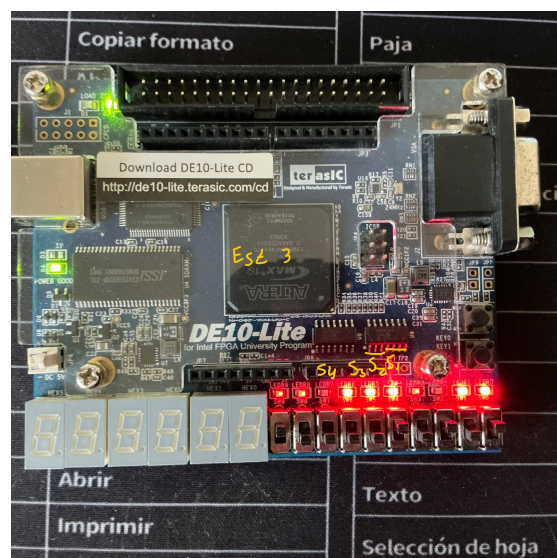


(b) S_1 con $B = 1$: se selecciona la liga verdadera hacia S_4 .

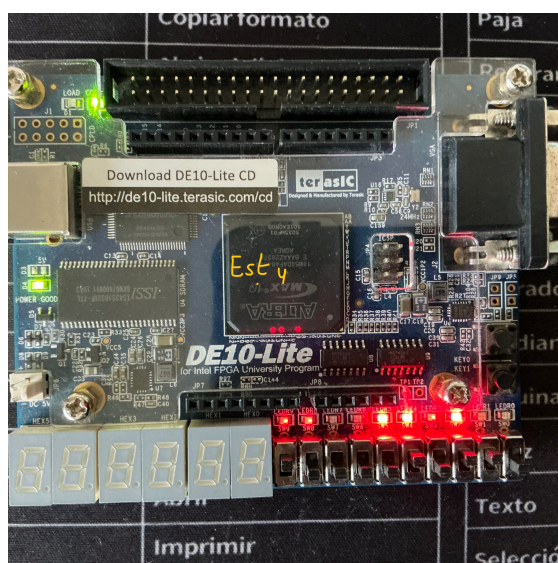
Figura 8: Verificación física de las dos ramas del estado S_1 .



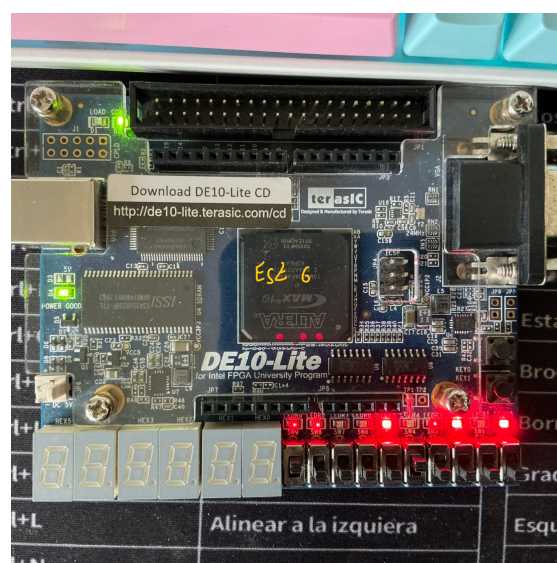
(a) Estado S_2 .



(b) Estado S_3 .



(c) Estado S_4 .



(d) Estado S_5 .

Figura 9: Evidencia en tarjeta de los estados S_2 , S_3 , S_4 y S_5 .

5. Conclusión

Percival Ulises Espinoza Matamoros. La construcción de la tabla de memoria permitió relacionar directamente cada bloque de la carta ASM con los campos de prueba, liga y salidas. La implementación confirmó que repetir las alternativas falsa y verdadera resuelve correctamente las transiciones incondicionales.

Oscar Iván Montiel Juárez. En esta práctica aplicamos los conceptos vistos en teoría sobre máquinas de estados, codificación binaria, memorias ROM y multiplexores. Al implementar el método en VHDL y observarlo en la tarjeta FPGA, se comprendió cómo una palabra de memoria puede contener tanto la decisión que se evalúa como las dos posibles respuestas del sistema.

Amy Valentina Veraza García. La simulación y las pruebas físicas permitieron verificar el funcionamiento de las ramas condicionadas por A , B y C , así como los estados de transición fija. La comparación entre la tabla, los módulos VHDL y los LEDs de la tarjeta ayudó a validar el diseño completo.